

UNIVERSIDAD DEL VALLE DE GUATEMALA

Bases de Datos 1 - Sección 10

Erick Marroquín



Proyecto 2

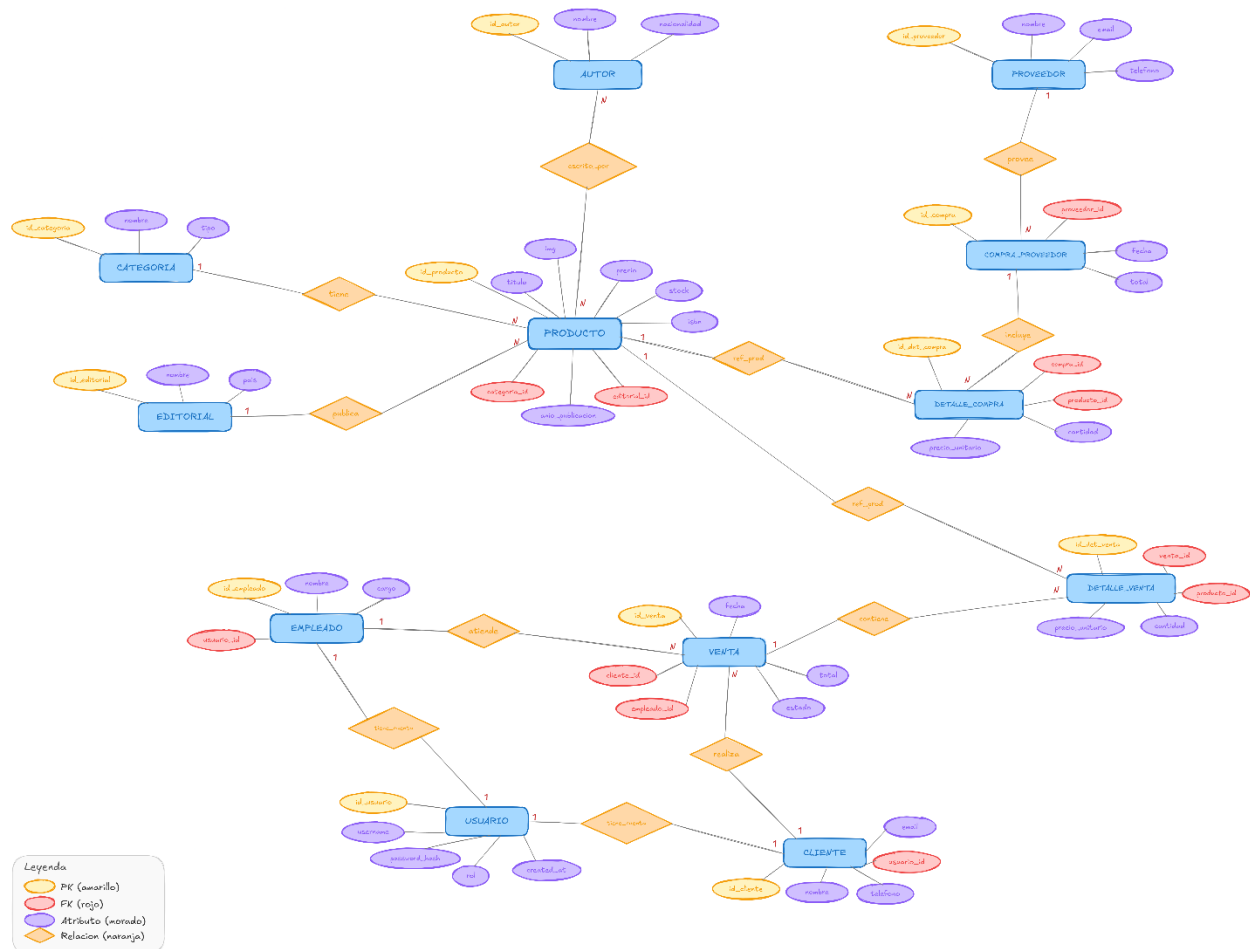
Angel Gabriel Sanabria Morales - 24725

Guatemala, 2026

Diagrama Entidad-Relación (DER)

El siguiente diagrama representa el modelo conceptual de la base de datos en notación Chen. Se identifican 13 entidades principales con sus atributos, claves primarias (amarillo), claves foráneas (rojo), atributos regulares (morado) y relaciones (naranja).

DER - Tienda de Libros y Mangas



Link al DER: <https://excalidraw.com/#json=CdKr9DikJFv3gneRuyg4J,dk6dN9OSB7cBesXS1E0G8Q>

Cardinalidades principales

- CATEGORIA (1) tiene PRODUCTO (N): una categoría agrupa muchos productos.
- EDITORIAL (1) publica PRODUCTO (N): una editorial publica muchos productos.
- AUTOR (N) escrito por PRODUCTO (N): relación muchos a muchos, resuelta con tabla puente producto autor (representada como relación en el DER; la tabla intermedia se crea en el DDL, validado con Erick vía Discord).
- PROVEEDOR (1) provee COMPRA_PROVEEDOR (N): un proveedor tiene muchas compras.
- COMPRA_PROVEEDOR (1) incluye DETALLE_COMPRA (N): cada compra tiene múltiples líneas.
- CLIENTE (1) realiza VENTA (N): un cliente puede hacer muchas ventas.
- EMPLEADO (1) atiende VENTA (N): un empleado atiende muchas ventas (nullable para compras web).
- VENTA (1) contiene DETALLE_VENTA (N): cada venta tiene múltiples productos.
- EMPLEADO (1) tiene cuenta USUARIO (1): relación uno a uno opcional.
- CLIENTE (1) tiene cuenta USUARIO (1): relación uno a uno opcional.

Modelo Relacional

A continuación, se presenta el esquema relacional documentado. Las claves primarias se indican como (PK) y las claves foráneas como (FK). Los atributos nullable se marcan explícitamente.

Tabla	Clave Primaria	Atributos
usuario	id_usuario (PK)	username, password_hash, rol, created_at
categoria	id_categoria (PK)	nombre, tipo, descripcion
editorial	id_editorial (PK)	nombre, pais, sitio_web
autor	id_autor (PK)	nombre, nacionalidad, fecha_nacimiento
producto	id_producto (PK)	titulo, descripcion, precio, stock, isbn, anio_publicacion, imagen_url, categoria_id (FK), editorial_id (FK), created_at
producto_autor	(producto_id, autor_id) PK compuesta	producto_id (FK), autor_id (FK)
proveedor	id_proveedor (PK)	nombre, contacto, telefono, email, direccion
compra_proveedor	id_compra (PK)	proveedor_id (FK), fecha, total, estado
detalle_compra	id_detalle (PK)	compra_id (FK), producto_id (FK), cantidad, precio_unitario
cliente	id_cliente (PK)	nombre, email, telefono, direccion, usuario_id (FK nullable), created_at
empleado	id_empleado (PK)	nombre, cargo, telefono, email, usuario_id (FK nullable), created_at
venta	id_venta (PK)	cliente_id (FK), empleado_id (FK nullable), fecha, total, estado
detalle_venta	id_detalle (PK)	venta_id (FK), producto_id (FK), cantidad, precio_unitario

Relaciones y Cardinalidades

Origen	Relación	Destino	Card.	Tipo FK
categoría	tiene	producto	1:N	NOT NULL
editorial	publica	producto	1:N	NOT NULL
autor	escrito_por	producto	N:M	tabla puente
proveedor	provee	compra_proveedor	1:N	NOT NULL
compra_proveedor	incluye	detalle_compra	1:N	NOT NULL
cliente	realiza	venta	1:N	NOT NULL
empleado	atiende	venta	1:N	NULLABLE
venta	contiene	detalle_venta	1:N	NOT NULL
usuario	tiene_cuenta	empleado	1:1	NULLABLE
usuario	tiene_cuenta	cliente	1:1	NULLABLE
producto	ref_prod	detalle_compra	1:N	NOT NULL
producto	ref_prod	detalle_venta	1:N	NOT NULL

Normalización hasta 3FN

Se demuestra que el esquema cumple con la Tercera Forma Normal (3FN). Para cada tabla se identifican las dependencias funcionales y se justifica la ausencia de dependencias parciales y transitivas.

Definiciones aplicadas

- 1FN: Todos los atributos son atómicos (no hay grupos repetitivos ni atributos multivaluados). Los campos tipo TEXT son valores únicos, no listas.
- 2FN: Todos los atributos no-clave dependen completamente de la clave primaria completa (relevante para producto_autor con PK compuesta).
- 3FN: No existen dependencias transitivas. Ningún atributo no-clave depende de otro atributo no-clave.

Análisis por tabla

- **Tabla: producto**
 - **Dependencias funcionales:** id_producto → titulo, descripcion, precio, stock, isbn, anio_publicacion, imagen_url, categoria_id, editorial_id
 - **Justificación 3FN:** Todos los atributos dependen completamente de la PK. No hay dependencias transitivas: categoria_id y editorial_id son FK que referencian entidades independientes, no atributos derivables.
- **Tabla: venta**
 - **Dependencias funcionales:** id_venta → cliente_id, empleado_id, fecha, total, estado
 - **Justificación 3FN:** Cada venta es un evento único. cliente_id y empleado_id son FK, no determinan otros atributos de venta.
- **Tabla: detalle_venta**
 - **Dependencias funcionales:** id_detalle → venta_id, producto_id, cantidad, precio_unitario
 - **Justificación 3FN:** precio_unitario se almacena en el detalle (no se deriva del producto) porque el precio puede cambiar en el tiempo. Sin dependencia transitiva.
- **Tabla: producto_autor**
 - **Dependencias funcionales:** (producto_id, autor_id) → (relación completa)
 - **Justificación 3FN:** PK compuesta. No hay atributos no-clave, por lo que está en BCNF trivialmente.
- **Tabla: usuario**
 - **Dependencias funcionales:** id_usuario → username, password_hash, rol, created_at
 - **Justificación 3FN:** username tiene restricción UNIQUE pero no determina otros atributos fuera de la PK. Sin transitividad.
- **Tabla: cliente**
 - **Dependencias funcionales:** id_cliente → nombre, email, telefono, direccion, usuario_id
 - **Justificación 3FN:** usuario_id es FK nullable (1:1 opcional). No hay dependencia entre nombre y usuario_id.
- **Tabla: detalle_compra**
 - **Dependencias funcionales:** id_detalle → compra_id, producto_id, cantidad, precio_unitario
 - **Justificación 3FN:** Misma lógica que detalle_venta. precio_unitario se almacena aquí porque el precio de compra al proveedor difiere del precio de venta al público y puede variar entre compras.

- **Tablas restantes (categoria, editorial, autor, proveedor, empleado, compra_proveedor)**
 - Todas estas tablas tienen PK simple (SERIAL) y atributos que dependen directa y únicamente de la PK. No existen dependencias parciales (PK simple) ni transitivas (los atributos son independientes entre sí). Ejemplo: en proveedor, el email no determina el teléfono ni la dirección; todos dependen solo de id_proveedor.
- **Caso especial: producto_autor (tabla puente)**
 - Esta tabla resuelve la relación N:M entre producto y autor. Su PK compuesta es (producto_id, autor_id). No tiene atributos no-clave, lo que la coloca trivialmente en BCNF. Si se necesitara agregar un atributo como rol_autor (autor principal, ilustrador, etc.), este dependería de la PK compuesta completa, manteniendo la 2FN.

DDL Completo

El DDL define 13 tablas con restricciones PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK y UNIQUE. Se incluye la extensión pgcrypto para hash de passwords, una VIEW para consulta de productos completos, y 5 índices explícitos sobre columnas de alta frecuencia de consulta.

DBMS: PostgreSQL 16

-- Tienda de Libros y Mangas - DDL
-- CC3088 Bases de Datos 1 - Proyecto 2

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

-- 1. Usuario

```
CREATE TABLE usuario (  
    id_usuario    SERIAL PRIMARY KEY,  
    username      VARCHAR(50) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    rol           VARCHAR(20) NOT NULL CHECK (rol IN ('admin', 'vendedor', 'cliente')),  
    created_at    TIMESTAMP DEFAULT NOW()  
);
```

-- 2. Categoria

```
CREATE TABLE categoria (  
    id_categoria  SERIAL PRIMARY KEY,  
    nombre        VARCHAR(100) NOT NULL,  
    tipo          VARCHAR(20) NOT NULL CHECK (tipo IN ('LIBRO', 'MANGA', 'REVISTA', 'COMIC')),  
    descripcion   TEXT  
);
```

-- 3. Editorial

```
CREATE TABLE editorial (  
    id_editorial  SERIAL PRIMARY KEY,  
    nombre        VARCHAR(100) NOT NULL,  
    pais          VARCHAR(50),  
    sitio_web     VARCHAR(255)  
);
```

-- 4. Autor

```
CREATE TABLE autor (  
    id_autor      SERIAL PRIMARY KEY,  
    nombre        VARCHAR(100) NOT NULL,  
    nacionalidad  VARCHAR(50),  
    fecha_nacimiento DATE  
);
```

-- 5. Producto

```
CREATE TABLE producto (  
    id_producto   SERIAL PRIMARY KEY,  
    titulo        VARCHAR(255) NOT NULL,  
    descripcion   TEXT,  
    precio        DECIMAL(10, 2) NOT NULL CHECK (precio > 0),  
    stock         INT NOT NULL DEFAULT 0 CHECK (stock >= 0),  
    isbn          VARCHAR(20) UNIQUE,  
    anio_publicacion INT,
```



```

        imagen_url    VARCHAR(500),
        categoria_id  INT NOT NULL REFERENCES categoria(id_categoria),
        editorial_id   INT NOT NULL REFERENCES editorial(id_editorial),
        created_at     TIMESTAMP DEFAULT NOW()
    );

```

-- 6. Producto-Autor (N:M)

```

CREATE TABLE producto_autor (
    producto_id  INT REFERENCES producto(id_producto) ON DELETE CASCADE,
    autor_id     INT REFERENCES autor(id_autor) ON DELETE CASCADE,
    PRIMARY KEY (producto_id, autor_id)
);

```

-- 7. Proveedor

```

CREATE TABLE proveedor (
    id_proveedor  SERIAL PRIMARY KEY,
    nombre        VARCHAR(100) NOT NULL,
    contacto      VARCHAR(100),
    telefono      VARCHAR(20),
    email         VARCHAR(100),
    direccion     TEXT
);

```

-- 8. Compra a Proveedor (cabecera)

```

CREATE TABLE compra_proveedor (
    id_compra     SERIAL PRIMARY KEY,
    proveedor_id  INT NOT NULL REFERENCES proveedor(id_proveedor),
    fecha         TIMESTAMP DEFAULT NOW(),
    total         DECIMAL(10, 2) NOT NULL DEFAULT 0,
    estado        VARCHAR(20) DEFAULT 'completada'
);

```

-- 9. Detalle de Compra

```

CREATE TABLE detalle_compra (
    id_detalle    SERIAL PRIMARY KEY,
    compra_id      INT NOT NULL REFERENCES compra_proveedor(id_compra) ON DELETE CASCADE,
    producto_id    INT NOT NULL REFERENCES producto(id_producto),
    cantidad       INT NOT NULL CHECK (cantidad > 0),
    precio_unitario DECIMAL(10, 2) NOT NULL CHECK (precio_unitario > 0)
);

```

-- 10. Cliente

```

CREATE TABLE cliente (
    id_cliente    SERIAL PRIMARY KEY,
    nombre        VARCHAR(100) NOT NULL,
    email         VARCHAR(100),
    telefono      VARCHAR(20),
    direccion     TEXT,
    usuario_id    INT UNIQUE REFERENCES usuario(id_usuario),
    created_at     TIMESTAMP DEFAULT NOW()
);

```

-- 11. Empleado

```

CREATE TABLE empleado (
    id_empleado   SERIAL PRIMARY KEY,
    nombre        VARCHAR(100) NOT NULL,

```

```

        cargo      VARCHAR(50),
        telefono    VARCHAR(20),
        email       VARCHAR(100),
        usuario_id  INT UNIQUE REFERENCES usuario(id_usuario),
        created_at   TIMESTAMP DEFAULT NOW()
    );

-- 12. Venta (cabecera)
CREATE TABLE venta (
    id_venta      SERIAL PRIMARY KEY,
    cliente_id    INT NOT NULL REFERENCES cliente(id_cliente),
    empleado_id   INT REFERENCES empleado(id_empleado),
    fecha         TIMESTAMP DEFAULT NOW(),
    total         DECIMAL(10, 2) NOT NULL CHECK (total >= 0),
    estado        VARCHAR(20) DEFAULT 'completada'
);

-- 13. Detalle de Venta
CREATE TABLE detalle_venta (
    id_detalle     SERIAL PRIMARY KEY,
    venta_id       INT NOT NULL REFERENCES venta(id_venta) ON DELETE CASCADE,
    producto_id    INT NOT NULL REFERENCES producto(id_producto),
    cantidad       INT NOT NULL CHECK (cantidad > 0),
    precio_unitario DECIMAL(10, 2) NOT NULL CHECK (precio_unitario > 0)
);

-- VIEW: vista_producto_completo
CREATE VIEW vista_producto_completo AS
SELECT
    p.id_producto, p.titulo, p.descripcion, p.precio, p.stock, p.isbn,
    p.anio_publicacion, p.imagen_url, p.categoria_id, p.editorial_id,
    c.nombre AS categoria, c.tipo,
    e.nombre AS editorial,
    STRING_AGG(a.nombre, ' ') AS autores
FROM producto p
JOIN categoria c ON p.categoria_id = c.id_categoria
JOIN editorial e ON p.editorial_id = e.id_editorial
LEFT JOIN producto_autor pa ON p.id_producto = pa.producto_id
LEFT JOIN autor a ON pa.autor_id = a.id_autor
GROUP BY p.id_producto, p.titulo, p.descripcion, p.precio, p.stock,
    p.isbn, p.anio_publicacion, p.imagen_url, p.categoria_id, p.editorial_id,
    c.nombre, c.tipo, e.nombre;

-- Indices
CREATE INDEX idx_producto_categoria ON producto(categoria_id);
CREATE INDEX idx_producto_editorial ON producto(editorial_id);
CREATE INDEX idx_venta_fecha ON venta(fecha);
CREATE INDEX idx_venta_cliente ON venta(cliente_id);
CREATE INDEX idx_compra_proveedor ON compra_proveedor(proveedor_id);

```

Restricciones implementadas

- PRIMARY KEY en todas las tablas (SERIAL o compuesta en producto_autor)

- FOREIGN KEY con integridad referencial (ON DELETE CASCADE en detalles y producto_autor)
- NOT NULL en campos obligatorios (titulo, precio, cantidad, etc.)
- CHECK constraints: precio > 0, stock >= 0, cantidad > 0, rol IN ('admin','vendedor','cliente')
- UNIQUE en username (usuario), isbn (producto), usuario_id en cliente y empleado (garantiza 1:1)

Vista definida

CREATE VIEW vista_producto_completo esta vista une producto + categoria + editorial + autores con STRING_AGG. Es consumida directamente por el backend para alimentar el listado de productos y el catálogo público.

Índices Justificados

Se definen 5 índices explícitos sobre columnas frecuentemente consultadas en operaciones de filtrado, búsqueda y generación de reportes. Todos están incluidos al final del archivo database/01-ddl.sql.

Índice	Columna	Justificación
idx_producto_categoria	producto(categoria_id)	Filtros frecuentes por categoría en la UI del catálogo. Los usuarios navegan por tipo (Manga, Libro, Comic). JOIN crítico para la vista de productos.
idx_producto_editorial	producto(editorial_id)	Búsquedas y filtros por editorial. Permite JOINS eficientes con la tabla editorial en vista_producto_completo.
idx_venta_fecha	venta(fecha)	Reportes de ventas por rango de fechas (ventas por mes, por semana). Query crítica para el dashboard de administración.
idx_venta_cliente	venta(cliente_id)	Historial de compras por cliente. Usado en la vista de perfil del cliente y en reportes de clientes frecuentes.
idx_compra_proveedor	compra_proveedor(proveedor_id)	Consultas de historial de compras por proveedor. Usado en reportes de cadena de suministro y vista de compras.

Datos de Prueba (Seed)

El script 02-seed.sql contiene datos realistas que superan los registros requeridos en las tablas operacionales principales. Los datos son coherentes entre sí y permiten probar todas las consultas SQL del proyecto.

Tabla	Registros	Descripción
usuario	25	admin, vendedores y clientes con passwords hashados via pgcrypto
categoria	25	Shonen, Seinen, Fantasia, Ciencia Ficción, Superheroes, Divulgación y más
editorial	25	Norma, Panini, Planeta, Salamandra, Ivrea y más
autor	25	Toriyama, Oda, Isayama, Rowling, Orwell, Asimov y más
producto	35	Mangas, libros, comics y revistas con ISBN, precio y stock reales
producto_autor	34	Relaciones producto-autor coherentes con los 35 productos
proveedor	25	Distribuidores en Guatemala con contacto y dirección
cliente	25	Mezcla de clientes con y sin cuenta de usuario
empleado	25	Vendedores, cajeros, bodegueros y supervisores
compra_proveedor	25	Compras a proveedores de enero a diciembre 2024
detalle_compra	25	Líneas de detalle para las compras a proveedor
venta	25	Ventas jun 2024 – ene 2025, con y sin empleado asignado
detalle_venta	42	Líneas de detalle para las 25 ventas

Arquitectura y Stack Técnico

Repositorio: <https://github.com/asanabria-2021067/proyecto2-db>

Estructura del monorepo

```
proyecto2-db/  
  assets/      # DER y recursos gráficos  
  frontend/    # Vue.js 3 + TypeScript  
  backend/     # Express.js + TypeScript  
  database/  
    01-ddl.sql  # Tablas, VIEW e índices  
    02-seed.sql # Datos de prueba  
  docker-compose.yaml  
  .env.example  
  README.md
```

Componentes

Capa	Tecnología	Rol
Frontend	Vue.js 3 + TypeScript + Axios	SPA con Composition API, rutas protegidas por rol
Backend	Express.js + TypeScript + pg	API REST con SQL raw (sin ORM)
Auth	bcryptjs + jsonwebtoken	Hashéo de passwords + JWT para sesiones (8h)
Base de datos	PostgreSQL 16	DBMS relacional con pgcrypto
Infra	Docker + docker-compose	3 servicios orquestados: db, backend, frontend

Despliegue

```
git clone https://github.com/asanabria-2021067/proyecto2-db.git  
cd proyecto2-db  
cp .env.example .env  
docker compose up --build
```

El proyecto se levanta con un único comando. Docker Compose orquesta los 3 servicios (db, backend, frontend) y ejecuta el DDL + seed automáticamente al iniciar el contenedor de PostgreSQL mediante un servicio db-init que espera el healthcheck de la base de datos.

Servicio	Puerto	Descripción
db	5432	PostgreSQL con esquema y datos seed
backend	3000	API REST Express
frontend	5173	Aplicación Vue 3 — http://localhost:5173

Credenciales de prueba

Usuario	Password	Rol
admin	admin123	admin
vendedor1	vend123	vendedor
cliente1	cli123	cliente